



RAT-Backend Dokumentation



RAT-Backend — Projektdokumentation

Remote Access Topology · Backend (REST-API)

DBI 2025–2026 · Schaffer Christof & Reichart Tobias



Inhaltsverzeichnis

1. [Projektbeschreibung](#)
2. [Team & Aufgabenverteilung](#)
3. [Architektur & Technologie](#)
4. [Datenbank-Design](#)
 - [Normalisierung \(1NF–3NF\)](#)
 - [ERM](#)
 - [RM-Übersicht \(PK/FK\)](#)
5. [API-Referenz](#)
 - [Endpunkte](#)
 - [HTTP-Statuscodes](#)
 - [Berechtigungssystem](#)
 - [Filterung, Sortierung & Pagination](#)
 - [Aggregation & Statistik](#)
6. [Logging](#)
7. [Sicherheit](#)
8. [Bedienungsanleitung \(Kurzfassung\)](#)
9. [Erfüllung der Anforderungen](#)
10. [KI-Nutzung](#)
11. [Projekttagebuch](#)
12. [Weiterführende Dokumente](#)



1. Projektbeschreibung

Remote Access Topology (RAT) ist eine Software, mit der man sein eigenes reales Netzwerk zentral überwachen, überprüfen und konfigurieren kann (z. B. über SNMP). Optisch ähnelt es Cisco Packet Tracer. Der Schwerpunkt liegt auf der automatischen Verbindung per **SSH, Telnet und FTP** (auf Mausklick) sowie auf **SNMP-Funktionen**.

Verwaltet werden:

User im Netzwerk, die auf verschiedene Geräte zugreifen, samt ihren Login-Daten auf den jeweiligen Geräten,

Rechte — nicht jeder User darf alles (rollenbasiertes Berechtigungssystem).

Dieses Repository ist das **Backend** (REST-API). Das **Frontend** ist eine C#-Desktop-Anwendung (WPF): <https://github.com/sumpfel/RAT-Client>.



2. Team & Aufgabenverteilung

Person	Git-Name	Verantwortung
Christof Schaffer	sumpfel	NetworkObject / NetworkObjectConnection / NetworkObjectInterface / NetworkObjectPermission / Login / SNMP, Berechtigungssystem
Tobias Reichart	Pir4t3141	Datenbank / Models, User- & UserSettings-Logik, Token-Login, Security

Gemeinsam: Planung und ERM.



3. Architektur & Technologie

- **Sprache:** Python 3
- **Web-Framework:** FastAPI (REST) + `fastapi-restful` (Class-Based-Views)
- **Server:** uvicorn (ASGI)
- **ORM / DB-Zugriff:** SQLAlchemy 2 (durchgehend parametrisierte Abfragen)
- **Validierung:** Pydantic 2 (DTOs / Schemas)
- **Auth:** JWT (`python-jose`) + Passwort-Hashing (`passlib` / `bcrypt`)
- **Datenbank:** SQLite (lokal, Standard) oder Cloud-DB über `DATABASE_URL`

Projektstruktur (Auszug):

```
src/  
├─ main.py           # App, Logging, Router-  
Registrierung, Default-Admin  
├─ database.py       # Engine (DATABASE_URL / .env),  
Session  
├─ models.py         # SQLAlchemy-Models (das Schema)  
├─ auth.py           # JWT + Passwort-Hashing  
├─ permissions.py    # Helfer für das  
Berechtigungssystem  
├─ routers/  
└─ statistics.py
```



4. Datenbank-Design

“

Vollständige, bearbeitbare Diagramme: `doc/assets/drawio/DB_Sketches_v2.drawio`. Detailfassung mit allen Bildern:
[Database_Sketches.md](#).

4.1 Normalisierung (1NF–3NF)

1NF — alle Attribute sind atomar: pro Spalte genau ein Wert, keine Listen oder Wiederholgruppen. Mehrfachbeziehungen (User↔Berechtigungen,

Gerät↔Interfaces) werden über eigene Zeilen in eigenen Tabellen abgebildet.

2NF — 1NF **und** volle funktionale Abhängigkeit vom Schlüssel. Jede Tabelle hat einen einspaltigen künstlichen PK (`id`), daher keine partiellen Abhängigkeiten. Die m:n-Beziehung User↔NetworkObject ist in

`NetworkObjectPermission` ausgelagert.

3NF — 2NF **und** keine transitiven Abhängigkeiten. Account-Einstellungen stehen in `UserSettings` (1:1), Geräte-Logins/SNMP hängen an der

Berechtigungszeile statt direkt am User oder Gerät.

4.2 ERM

`User` **1:1** `UserSettings`

`User` **m:n** `NetworkObject` (aufgelöst über

`NetworkObjectPermission`, Level 0–4)

`NetworkObject` **1:n** `NetworkObjectInterface`

`NetworkObjectInterface` **n:1** `NetworkObjectConnection` (zwei Interfaces = ein Kabel)

`NetworkObjectPermission` **1:n** `Login` und **1:n** `SNMPSettings`

4.3 RM-Übersicht

“

unterstrichen = Primärschlüssel (PK) · *kursiv* = Fremdschlüssel (FK)

User: id, username (unique), password (bcrypt), is_admin, canCreate **UserSettings:**

id, *user_id* → User, zoom, showPorts, showInterfaces **NetworkObject:** id, name

(unique), type, x, y, os, cpu, gpu, ram, specs **NetworkObjectInterface:** id,

network_object_id → NetworkObject, *network_object_connection_id* →

NetworkObjectConnection (nullable), name, max_speed, is_up, ipv4, ipv6,

ipv4_subnet_mask, ipv6_prefix_length, ipv4_gateway **NetworkObjectConnection:**

id, name, speed, type, note **NetworkObjectPermission:** id, *user_id* → User,

network_object_id → NetworkObject, permissions (0–4) **Login:** id,

network_object_permission_id → NetworkObjectPermission, port, type, username, password
SNMPSettings: *id*, *network_object_permission_id* → NetworkObjectPermission, read_community, write_community

Die ausführlichen Spalten/Typ-Tabellen stehen in [Database_Sketches.md](#).



5. API-Referenz

Basis-URL (lokal): `http://127.0.0.1:8000` · interaktive Doku: `/docs`
(Swagger UI). Alle Endpunkte außer Login/Register benötigen `Authorization: Bearer <JWT>`.

5.1 Endpunkte

Methode & Pfad	Zweck
<code>POST /user/login</code>	JWT holen (OAuth2-Form-Login)
<code>POST /user/register</code> · <code>GET /user/</code> · <code>GET /user/me</code>	User anlegen / auflisten / aktuell
<code>PUT /user/{id}</code> · <code>DELETE /user/{id}</code>	User bearbeiten / löschen (Admin-Regeln)
<code>GET/POST/PUT/DELETE /networkObject/</code>	Geräte (CRUD, Filter/Sort/ Pagination)
<code>GET/POST/PUT/DELETE / networkObjectInterface/</code>	Interfaces
<code>GET/POST/PUT/DELETE / networkObjectConnection/</code>	Verbindungen (Kabel)

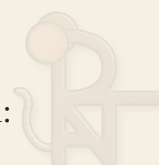
Methode & Pfad	Zweck
GET/POST/PUT/DELETE / networkObjectPermission/	Zugriffsrechte pro Objekt
GET/POST/PUT/DELETE /login/ · / snmpSettings/	Geräte-Logins / SNMP
GET /user/settings/ · PUT /user/ settings/	UI-Einstellungen pro User
GET /statistics/summary · /objects-by- type · /connections-by-type · / interfaces-per-object	Aggregation / Statistik

5.2 HTTP-Statuscodes

Code	Bedeutung
200	erfolgreiches GET / PUT / DELETE
201	erfolgreiches Anlegen (POST)
400	Validierungsfehler / Passwort-Policy / ungültige Berechtigungsänderung
401	falsche Login-Daten oder ungültiges/abgelaufenes Token
403	angemeldet, aber zu wenig Rechte
404	unbekannte ID (oder verborgenes Objekt)
409	Konflikt: Username oder NetworkObject-Name existiert bereits

5.3 Berechtigungssystem

Pro Gerät hält `NetworkObjectPermission.permissions` ein Level:



Level	Name	Darf
0	Hidden	Gerät & Verbindungen gar nicht sehen
1	See	Gerät + Interfaces sehen; eigene Logins/SNMP hinterlegen
2	Edit	Interfaces & Einstellungen ändern (nicht löschen)
3	Admin	Edit + Rechte von Usern mit <i>niedrigeren</i> Rechten vergeben (0–2)
4	Owner	Admins ändern, Owner vergeben/entziehen, Gerät löschen

Globale Admins (`is_admin`) haben implizit Owner auf allem. Verborgene Objekte werden mit `404` (statt `403`) beantwortet, damit ihre Existenz nicht durchsickert.

5.4 Filterung, Sortierung & Pagination

Alle Listen-Endpunkte akzeptieren optionale Query-Parameter (Defaults lassen bestehende Aufrufe unverändert):

- `networkObject`: `name` (Suche), `type`, `sort_by`, `order`, `limit`, `offset`
- `networkObjectInterface`: `network_object_id`, `name`, `is_up`, `limit`, `offset`
- `networkObjectConnection`: `name`, `type`, `min_speed`, `sort_by`, `order`, `limit`, `offset`
- `user`: `username`, `is_admin`, `limit`, `offset`

Filter nutzen gebundene `LIKE` /Gleichheits-Parameter; die Sortierung verwendet eine **Whitelist** erlaubter Spalten (kein beliebiger String erreicht das SQL).



5.5 Aggregation & Statistik

Read-only Endpunkte mit `GROUP BY` + `COUNT` / `SUM` / `AVG` / `MIN` / `MAX` :

- `GET /statistics/summary` — Anzahl Objekte/Interfaces/Connections + AVG/SUM/MAX/MIN der Connection-Speeds
- `GET /statistics/objects-by-type` — `GROUP BY NetworkObject.type`, `COUNT`
- `GET /statistics/connections-by-type` — `GROUP BY type`, `COUNT` + `AVG` / `SUM` Speed
- `GET /statistics/interfaces-per-object` — `GROUP BY` Gerät, `COUNT` der Interfaces

Alle respektieren die Sichtbarkeit: ein normaler User sieht nur Statistiken über Objekte, auf die er mindestens See(1) hat; Admins sehen alles.



6. Logging

Pflicht erfüllt mit Python's `logging`-Modul (`src/main.py`):

- **INFO** für jeden eingehenden Request (Methode, Pfad, Statuscode) über eine Middleware.
 - **ERROR** für unerwartete Fehler über einen globalen Exception-Handler, der zugleich ein sauberes `500` zurückgibt (kein Stacktrace / keine DB-Internas im Response).
- Validierungsfehler werden als **WARNING** geloggt.
- Ausgabe in **Konsole und Datei** `api.log`.



7. Sicherheit

Passwörter werden mit bcrypt gehasht (nie im Klartext gespeichert).

Passwort-Policy: ≥ 8 Zeichen, mind. ein Buchstabe und eine Ziffer (Server- und Client-seitig).



- **SQL-Injection:** ausgeschlossen — sämtlicher DB-Zugriff läuft über das
- SQLAlchemy-ORM mit gebundenen Parametern; Filter nutzen `ILIKE` / Gleichheit auf Spaltenobjekten, Sortierung eine Spalten-Whitelist. Kein string-konkateniertes SQL.

Auth: JWT mit Ablaufzeit; ungültiges/abgelaufenes Token → `401`.

Default-Admin: bei frischer DB `admin` / `admin` — **nach erstem Login ändern.**

Offen (bekannt): `SECRET_KEY` ist in `auth.py` noch hart kodiert; sollte produktiv in eine `.env` ausgelagert werden.



8. Bedienungsanleitung (Kurzfassung)

Schnellstart Windows (eine Zeile):

```
git clone https://github.com/sumpfel/RAT-Backend.git &&
cd RAT-Backend && set HOST=0.0.0.0 && set
PORT=8080 && run.bat
```

Linux / macOS:

```
git clone https://github.com/sumpfel/RAT-Backend.git &&
cd RAT-Backend && ./run.sh
```

Die Run-Skripte legen `.venv` an, installieren `src/requirements.txt` und starten den Server. Danach: API unter `http://127.0.0.1:8000`, Swagger unter `/docs`.

Beispiel — Login & ein Gerät anlegen:

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/user/login
-d "username=admin&password=admin" \
| python3 -c "import sys,json;print(json.load(sys.stdin)
['access_token'])")

curl -X POST http://127.0.0.1:8000/networkObject/ \
```

```
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"name":"PC-1","type":"PC","x":10,"y":20,"os":"Linux","cpu":"i5","gpu":'}
```

Vollständige Anleitung mit allen Beispiel-Requests: [Bedienungsanleitung.md](#).



9. Erfüllung der Anforderungen

Anforderung	Status	Wo
Aggregationsendpunkt (GROUP BY + COUNT/SUM/AVG)	✓	<code>routers/</code> <code>statistics.py</code>
HTTP-Statuscodes korrekt (200/201/400/404/409/401)	✓	alle Router
SQL-Injection-Schutz (parametrisiert)	✓	SQLAlchemy-ORM durchgehend
Logging (INFO + ERROR, Konsole + <code>api.log</code>)	✓	<code>main.py</code>
Erweiterte Filterung (Query-Parameter)	✓	GET-Listen
Pagination (<code>limit</code> / <code>offset</code>)	✓	GET-Listen
Cloud-Datenbank	vorbereitet	<code>database.py</code> (<code>DATABASE_URL</code> / <code>.env</code>)
Erweiterte Aggregation/Statistik	✓	<code>routers/</code> <code>statistics.py</code>
Projekttagebuch (wer/wann/was)		Projekttagebuch.md

Anforderung	Status	Wo
requirements.txt vollständig		src/ requirements.txt
Doku als PDF		doc/pdf/



10. KI-Nutzung

Alle KI-Einsätze sind in [AI_usage.md](#) dokumentiert (Modell, Datum, Prompt, betroffene Dateien). Im Code sind die Stellen mit `KI Claude <KI-N> ... KI` `END <KI-N>` markiert; `N` verweist auf den jeweiligen Eintrag in `AI_usage.md`.



11. Projekttagbuch

Das laufend geführte Projekttagbuch (wer / wann / was, aus den Git-Commits, im Frontend-Stil ADDED/FIXED/CHANGED) steht in [Projekttagbuch.md](#).



12. Weiterführende Dokumente

Dokument	Inhalt
Bedienungsanleitung.md	Start der API + alle Beispiel-Requests
Database_Sketches.md	Normalisierung, ERM, RM mit Bildern & Spalten
Project_Planning.md	ursprüngliche Projektplanung

Dokument	Inhalt
Projekttagbuch.md	Projekttagbuch
AI_usage.md	dokumentierte KI-Nutzung
DB_Sketches_v2.drawio	bearbeitbares ERM/RM-Diagramm



RAT-Backend · DBI 2025–2026 · Schaffer & Reichart · rats theme

